INTESI GROUP

Progetto – User Managment Service

Progettazione e sviluppo di un servizio back-end scalabile in Java su architettura a microservizi

COPYRIGHT DISCLAIMER

Tutti i Contenuti (testi, immagini, specifiche tecniche e altro) del presente documento sono **proprietà esclusiva e riservata di Intesi Group** e/o dei suoi aventi causa e/o di terzi soggetti ove indicati, e sono protetti dalle vigenti norme nazionali ed internazionali in materia di proprietà Intellettuale e/o Industriale.
É pertanto vietato utilizzare in qualsiasi modalità (a mero titolo esemplificativo, modificare, copiare, riprodurre, distribuire, trasmettere o diffondere) i suddetti Contenuti senza la previa autorizzazione scritta da parte del Titolare e/o dagli aventi diritto che se ne riservano espressamente ogni forma di riproduzione ed utilizzo. Ogni violazione sarà perseguita a norma di legge.

All Contents (texts, images, technical specifications and more) of this document are the **exclusive and reserved property of Intesi Group** and/or its successors in title and/or third parties where indicated, and are protected by current national and international regulations in intellectual and/or industrial property matters.
It is therefore forbidden to use in any way (by way of example only, modify, copy, reproduce, distribute, transmit or disseminate) the aforementioned Contents without the prior written authorization of the Owner and/or those entitled who expressly reserve any form of reproduction and use it. Any violation will be prosecuted according to the law.

Obiettivo

Realizzare un servizio backend enterprise-grade per la gestione degli utenti di un prodotto, progettato secondo principi di scalabilità, manutenibilità e solidità, tipici di un sistema a microservizi.

Il focus non è solo sulla funzionalità, ma sulla qualità architetturale, sulle scelte tecniche e sulla chiarezza del codice.

Dominio Applicativo

Entità Utente

Ogni utente è caratterizzato dai seguenti attributi:

- id
- username
- email (univoca, non modificabile)
- codiceFiscale (valido e univoco)
- nome
- cognome
- status (es. ACTIVE, DISABLED, DELETED)
- createdAt, updatedAt

Ad ogni utente possono essere associati uno o più ruoli applicativi:

- OWNER
- OPERATOR
- MAINTAINER
- DEVELOPER
- REPORTER

È richiesta una modellazione corretta del dominio, con particolare attenzione a vincoli, cardinalità e integrità dei dati.

Requisiti Tecnici

Stack Tecnologico

- Java + Spring Boot
- Architettura a layer (Controller, Service, Repository)
- Utilizzo di DTO e mapping esplicito
- Persistenza dati su database a scelta, motivando la scelta
- Documentazione API (OpenAPI / Swagger)

Persistenza e Modellazione Dati

- Modellazione dello schema con:
  - vincoli di unicità
  - gestione delle relazioni utente–ruoli
  - gestione dello stato dell'utente (soft delete consigliato)
- Attenzione a:
  - Indicizzazione
  - query efficienti
  - coerenza dei dati

API REST – Requisiti Funzionali

Requisiti Obbligatori

Implementare API REST per:

1. Recupero lista utenti (con paginazione)
2. Recupero dettaglio di un singolo utente
3. Creazione di un utente con ruoli associati
4. Modifica dei dati utente e/o dei ruoli
5. Disabilitazione o cancellazione logica di un utente

Le API devono:

- rispettare le best practice REST
- gestire correttamente gli errori (HTTP status + payload chiaro)
- validare gli input (es. email, codice fiscale)

Aspetti Architetturali (fondamentali)

Il progetto deve dimostrare attenzione a:

- Separazione delle responsabilità
- Codice manutenibile e testabile
- Chiarezza delle dipendenze
- Gestione consistente delle eccezioni
- Logging significativo

Nel README o in un file dedicato è richiesto spiegare:

- le scelte architetturali
- eventuali trade-off

Requisiti Avanzati (Plus – ma molto valutati)

Sicurezza & Identity Management (Plus fortemente valorizzato)

- Autenticazione tramite JWT
- Autorizzazione RBAC
- Protezione degli endpoint in base al ruolo
- Possibilità di limitare i dati restituiti in base al ruolo (es. campi mascherati)
- Integrazione con un sistema di Identity & Access Management, preferibilmente Keycloak

Eventi Asincroni

- Pubblicazione di un evento asincrono alla creazione di un utente
- Tecnologia a scelta (Es: RabbitMQ)
- Descrivere l'evento e il suo scopo

Qualità e Testing

- Test unitari sui servizi
- Test di integrazione sulle API principali
- Uso consapevole dei test (non serve coverage totale)

DevOps & Runtime

- Dockerizzazione dell'applicazione
- Docker / docker-compose per:
    - database
    - avvio locale del progetto
- (Plus) configurazione base di pipeline CI/CD (anche solo descritta)

Requisiti di Consegna

- Repository GitHub pubblico o condiviso
- README.md che includa:
    - descrizione del progetto
    - istruzioni di build ed esecuzione
    - spiegazione delle scelte tecniche
- Codice leggibile, commentato dove necessario

Criteri di Valutazione

Il progetto verrà valutato su:

- Qualità del design
- Scelte architetturali
- Pulizia del codice
- Gestione della complessità
- Capacità di ragionare come su un sistema reale